# Models of Higher Brain Function

Lecture Notes and Python Modeling Workspace

Arjun Puri

Bernstein Center for Computational Neuroscience, Berlin

Summer 2026

# Contents

**Preface** v

**Notation** vii

**I Foundations** 1

**1 Course Overview and Modeling Strategy** 3

1.1 What makes a model useful? . . . 3
1.2 A common abstraction stack . . . 3
1.3 Code organization . . . 4
1.4 Next steps . . . 5

**II Perception And Representation** 7

**2 Visual Perception and Deep Learning** 9

2.0.1 Why vision belongs in a course on higher brain function . . . 9
2.0.2 Invariant object recognition . . . 10
2.0.3 The ImageNet turning point . . . 11
2.0.4 What deep networks compute . . . 12
2.0.5 Deep networks as models of ventral visual cortex . . . 13
2.0.6 Deep networks as experimental tools . . . 17
2.0.7 What deep networks still miss . . . 18
2.0.8 Summary of concepts introduced in Lecture 1 . . . 18
2.0.9 Conceptual takeaways from Lecture 1 . . . 19

**Mathematical Synthesis** 21

**Bibliography** 25

# Preface

This directory contains the lecture notes for *Models of Higher Brain Function*. It uses the same book-style LaTeX structure as Models of Neural Systems, but the underlying template has been extracted into a reusable course-notes stack so the notes can grow independently. The opening scaffold is intentionally compact: one overview chapter, a clean bibliography, and a minimal foundation for later material on cognition, memory, decision making, and cortical computation.

*Berlin, April 2026*

# Notation

## Dynamical systems

| Symbol | Meaning |
|---|---|
| $x_t$ | State at time $t$ |
| $u_t$ | Input or control signal |
| $f(\cdot)$ | State transition or update map |
| $\tau$ | Characteristic time constant |

## Representations and learning

| Symbol | Meaning |
|---|---|
| $\mathbf{h}$ | Hidden representation |
| $\mathbf{W}$ | Weight matrix |
| $L$ | Objective or loss function |
| $\nabla L$ | Gradient of the objective |

## Probability and inference

| Symbol | Meaning |
|---|---|
| $p(z \mid x)$ | Posterior distribution |
| $\mathbb{E}[\cdot]$ | Expectation |
| $\mathrm{Var}[\cdot]$ | Variance |
| $\arg\max$ | Optimization operator |

Part I

# Foundations

# 1

# Course Overview and Modeling Strategy

Higher brain function is a modeling problem before it is a tooling problem. Memory, planning, inference, abstraction, and decision making are all latent processes: we rarely observe them directly, but instead infer them from behavior, neural activity, and task structure. A useful course repository therefore needs space for both careful conceptual writing and executable model-building.

This scaffold is designed for exactly that. The LaTeX notes can hold the conceptual arc of the course, while the Python package can accumulate simulations, optimization experiments, and quick probes of candidate mechanisms.

## 1.1 What makes a model useful?

A model of higher brain function should do more than fit observations. It should compress behavior into a tractable set of assumptions, explain why certain error patterns appear, and suggest new experiments or interventions. In cognitive and systems neuroscience, this usually means balancing three goals:

- computational clarity,
- biological plausibility,
- empirical testability.

> Practical heuristic
>
> If a model explains everything equally well, it usually explains nothing. Useful models make sharp commitments about mechanisms, variables, and expected failures.

## 1.2 A common abstraction stack

Many courses on higher brain function move across a recurring abstraction ladder: behavior and task structure, latent cognitive variables, circuit or algorithmic mechanisms, and normative or optimization principles.

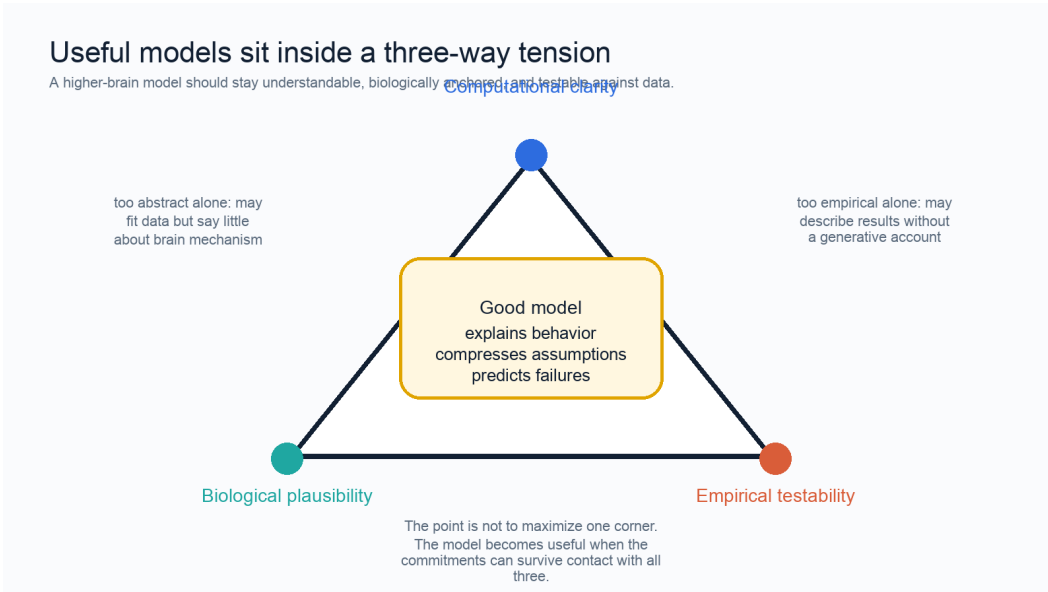


Figure 1.1: Why useful models live in tension. Strong higher-brain models stay interpretable, stay anchored to plausible mechanisms, and still make contact with measurable data.

That ladder connects classical cognitive models to modern neural-network and dynamical-systems viewpoints. The literature on reinforcement learning, attractor dynamics, and recurrent computation makes clear that representation, dynamics, and objective functions must be studied together [DA01; ORe+06; Wan08].

Generic state-space view

$$x_{t+1} = f(x_t, u_t; \theta)$$

This form is broad enough to include recurrent neural networks, drift-diffusion style updates, working-memory attractors, and many cognitive-state models.

## 1.3 Code organization

The Python scaffold is deliberately simple. It gives the repository a real package layout immediately, so simulation code and reusable utilities do not end up trapped in ad hoc notebooks. Recommended directions for growth include simulation modules for canonical

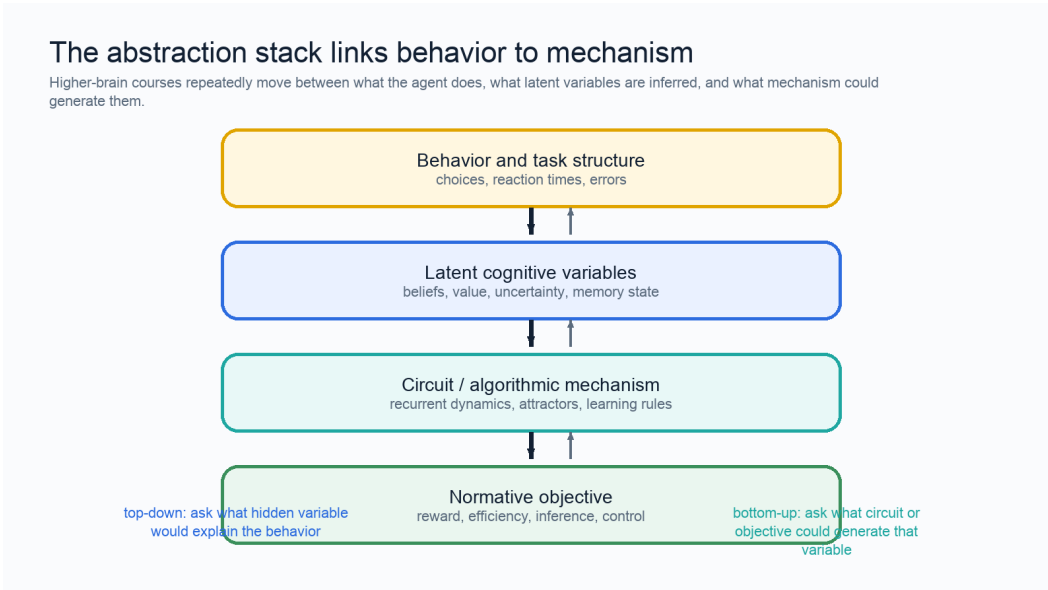


Figure 1.2: The abstraction ladder used throughout the course. Explanations move downward from observed behavior to hidden variables and mechanism, then upward again to ask whether the mechanism reproduces the behavior.

cognitive tasks, optimization utilities for fitting simple models, plotting helpers for trajectories and phase portraits, and notebook-based reading replications.

## 1.4 Next steps

Add one chapter per conceptual block, keep derivations in the notes, and let the source package accumulate the minimal reusable code needed to reproduce course figures and experiments. Over time, the repository can become both a reading guide and a computational playground.

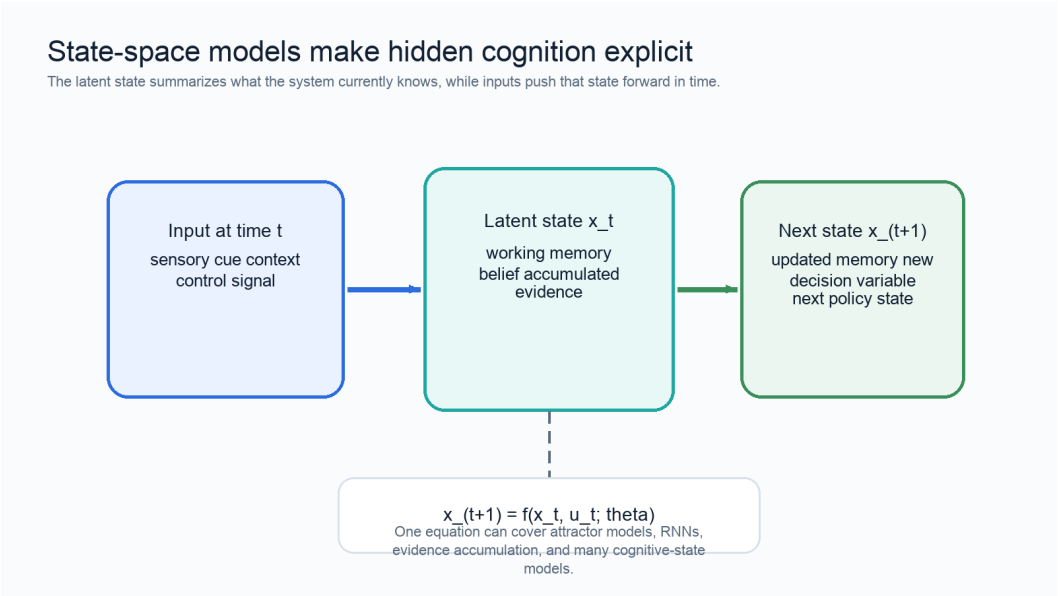


Figure 1.3: State-space intuition. The hidden state carries forward what the system currently "knows," inputs perturb that state, and the update rule determines the next internal configuration.

**Part II**

# Perception And Representation

# 2

# Visual Perception and Deep Learning

These notes cover the opening perception lecture of *Models of Higher Brain Function*. The lecture uses vision as the first concrete example of a higher-brain computation: the sensory input is high-dimensional, noisy, and ambiguous, but behavior requires a stable internal description of objects and scenes. Deep neural networks enter the lecture not as a fashionable add-on, but as a way to make the representation problem explicit. They give us a concrete family of models that can solve large-scale object recognition tasks and can then be compared with the ventral visual stream.

### 2.0.1 Why vision belongs in a course on higher brain function

The lecture opens with a basic question: what does a visual system need to know about the world? At the level of behavior, the answers are simple enough:

- what is out there,
- where it is,
- and what it is likely to do next.

Those questions already show why vision is not just a matter of registering light intensity. A retinal image is only a pattern of local measurements. Perception requires the system to organize that pattern into useful latent variables such as object identity, position, motion, and behavioral relevance.

#### Why this problem is harder than it feels

One theme from the lecture is Moravec's paradox: tasks that feel effortless to humans often turn out to be extremely demanding computationally. Recognizing a cup after it has moved, rotated, changed scale, or appeared under different illumination feels trivial in everyday life, but it is difficult because the raw sensory input changes enormously while the object identity stays the same.

This is why visual perception is treated here as a problem of inference rather than mere detection. The brain must move from a changing image to a useful internal representation of the world.

### 2.0.2 Invariant object recognition

The core challenge identified in the lecture is *invariant object recognition*. A useful visual system should keep track of what an object is even when many superficial aspects of the image change.

#### Why pixels are a bad format for recognition

The lecture follows the argument from DiCarlo and Cox that the raw image is a poor coordinate system for object recognition. Pixel values are highly sensitive to:

- translation across the retina,
- viewing angle,
- distance and scale,
- illumination,
- occlusion,
- and background clutter.

So two pictures of the same object can be very far apart in pixel space, while pictures of different objects may overlap heavily in their low-level image statistics. The slides describe this by saying that object manifolds are "hopelessly tangled up" in the original sensory coordinates.

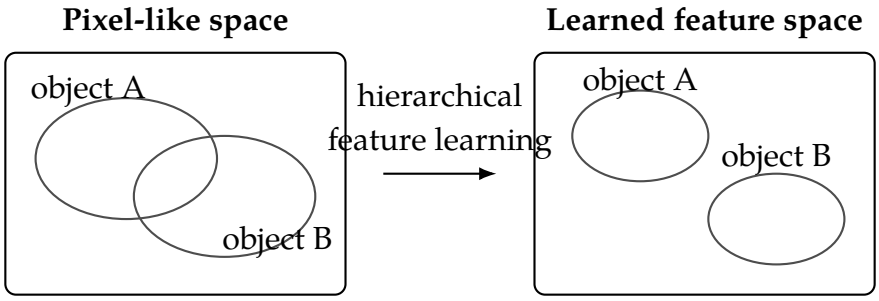


Figure 2.1: The geometric picture emphasized in the lecture. In raw image coordinates, object manifolds are tangled together. A good representation transforms them into a space where object identity is easier to separate.

#### What makes a representation useful

The lecture's answer is that a "good" neural representation should vary gradually with nuisance variables such as position, illumination, or angle, while making object identity easier to read out. In other words, recognition becomes easier when the representation performs a kind of *disentangling*: factors that should be treated as irrelevant for identification have a smaller effect on the code than they do in the original image.

This idea is important because it shifts the focus away from raw accuracy and toward representational geometry. The real question is not only whether the system labels an image correctly, but whether it transforms sensory input into coordinates that make the task easy for later stages.

### 2.0.3 The ImageNet turning point

The lecture then turns from the abstract recognition problem to the modern deep-learning solution. The key historical moment is the ImageNet era. ImageNet supplied a large-scale benchmark with roughly fifteen million labeled images spanning one thousand categories. This mattered because it forced models to solve recognition at a realistic visual scale rather than on small toy datasets.

#### Why AlexNet changed the conversation

AlexNet's 2012 ImageNet result is presented as the turning point. Its top-5 error was 16.4%, while the next-best system remained at 26.2%. The significance of that result was not just the leaderboard gap. It showed that a deep, trainable convolutional network could learn useful visual features directly from data instead of depending mainly on hand-engineered descriptors.

The lecture also describes the "magic sauce" behind this success. Several ingredients had to come together at once:

- large labeled datasets,
- data augmentation,
- substantial computing power,
- convolution and weight sharing,
- rectified linear units,
- pooling and downsampling,

- and architectures deep enough to build hierarchical features.

So deep learning did not succeed because of one isolated trick. It succeeded because data, optimization, hardware, and architecture started to reinforce one another.

#### From AlexNet to ResNet

The lecture places AlexNet in a broader architectural sequence. VGG-style networks increased depth and regularized the use of small convolutions. ResNets then made very deep models trainable by adding skip connections. Instead of asking each block to learn an entirely new transformation, a residual block can learn a correction to the identity map. This helps with vanishing gradients and with the degradation in performance that appears when very deep plain networks become hard to optimize.

The slides make this concrete with the ImageNet benchmark again: ResNets reached depths above one hundred layers and pushed error rates below typical human performance on that dataset. The lecture's point is not that ImageNet is a perfect model of human vision, but that these models had become competent enough that their internal representations became scientifically interesting.

### 2.0.4 What deep networks compute

At the mathematical level, the lecture describes a deep network as a concatenation of linear-nonlinear transformations. A single layer takes an input vector $\mathbf{x}$, applies an affine transformation, and then a nonlinearity:

$$\mathbf{y} = \phi(W\mathbf{x} + \mathbf{b}) .$$

Here $W$ is the weight matrix, $\mathbf{b}$ is a bias term, and $\phi$ is a nonlinear activation function.

#### Why nonlinearity matters

If every layer were purely linear, then stacking many layers would still amount to one overall linear transformation. Depth would not buy us much representational power. Nonlinearity is what lets the network bend and reshape the input space so that later layers can express more abstract and more separable features.

The lecture's qualitative story is straightforward:

- early layers detect simple local patterns such as oriented edges or contrast changes,
- intermediate layers combine those into textures or parts,

- later layers become more useful for category-level distinctions.

This staged abstraction is exactly why deep networks became plausible models of a hierarchical sensory pathway.

#### Why convolution is useful for images

Convolutional layers add two inductive biases that fit images well.

**Local receptive fields.** Each filter only sees a small image patch at a time. This reflects the fact that nearby pixels are strongly related and that many informative visual structures are local.

**Weight sharing.** The same filter is applied at many spatial positions. This makes it possible to detect the same local pattern wherever it appears and reduces the total number of parameters that must be learned.

Pooling and striding then expand the effective receptive field and make the representation less sensitive to small translations. This does not create perfect invariance, but it pushes the representation in the right direction for object recognition.

#### Backpropagation and the credit-assignment problem

Training is described in the lecture through backpropagation and stochastic gradient descent. A loss function measures the discrepancy between the network output and the target label, and gradients are propagated backward through the chain of nonlinear functions. A typical parameter update has the form

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w},$$

where $\mathcal{L}$ is the loss and $\eta$ is the learning rate.

This solves the engineering problem of *credit assignment*: it tells the system how each parameter contributed to the final error. The lecture is careful, however, not to overinterpret this as a biological mechanism. Exact backpropagation, large labeled datasets, and perfectly shared convolutional filters are all biologically questionable. So deep networks are useful models of visual computation without being literal copies of cortical learning.

### 2.0.5 Deep networks as models of ventral visual cortex

Once deep networks became good at object recognition, the next question was whether their internal representations looked at all like those found in ventral visual cortex. The lecture

emphasizes that the comparison should not be made unit by unit. A hidden unit in a trained network is not expected to correspond neatly to one biological neuron, and different random initializations of the same architecture need not preserve any unique one-to-one matching.

#### Why compare representational geometry

The more useful comparison is at the level of *representational similarity*. For a collection of stimuli, one can compute a response pattern for each stimulus and then ask how dissimilar those patterns are from one another. This yields a representational dissimilarity matrix:

$$\mathrm{RDM}_{ij} = 1 - \mathrm{corr}(r_i, r_j).$$

Here $r_i$ and $r_j$ denote the response vectors evoked by stimuli $i$ and $j$.

The important point is that two systems can have similar representational geometry even if their individual units are not aligned. This is why representational similarity analysis became so useful in comparing human IT, monkey IT, and deep-network hidden layers.

#### The RSA construction

Suppose the experiment presents $n$ stimuli. For one system, such as a cortical area or a neural-network layer, collect the responses into a matrix

$$R = \begin{bmatrix} r_1^\top \\ r_2^\top \\ \vdots \\ r_n^\top \end{bmatrix} \in \mathbb{R}^{n \times p}.$$

The row $r_i \in \mathbb{R}^p$ is the population response to stimulus $i$. In a brain dataset, the $p$ dimensions might be voxels, recording sites, or neurons. In a network, they might be hidden units or feature channels.

RSA then replaces the response matrix by an $n \times n$ matrix of pairwise dissimilarities:

$$D_{ij} = d(r_i, r_j).$$

The function $d$ defines what kind of representational difference is being measured. The most common choice in this lecture is correlation distance,

$$d_{\mathrm{corr}}(r_i, r_j) = 1 - \frac{(r_i - \bar{r}_i \mathbf{1})^\top (r_j - \bar{r}_j \mathbf{1})}{\|r_i - \bar{r}_i \mathbf{1}\|_2 \, \|r_j - \bar{r}_j \mathbf{1}\|_2}.$$

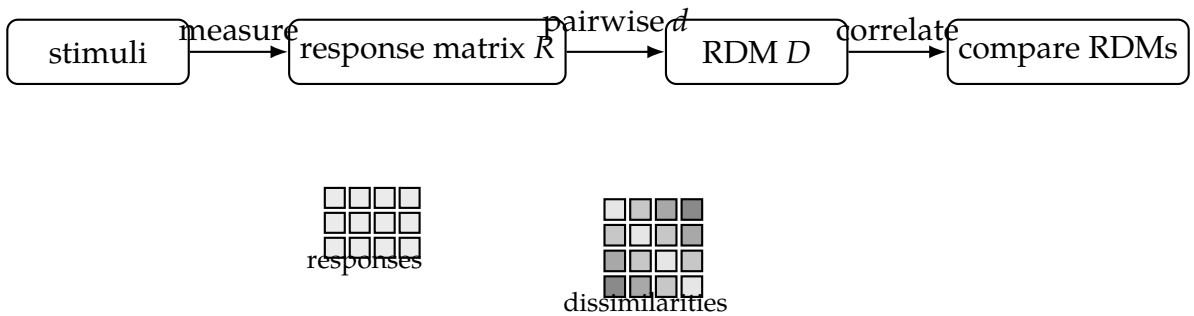


Figure 2.2: Representational similarity analysis turns high-dimensional response patterns into a matrix of stimulus-by-stimulus distances. The comparison between brain and model happens at the level of these distance matrices.

This quantity ignores the overall response scale and asks whether the two response *patterns* have the same shape across units. A small value means that the two stimuli evoke similar population patterns; a large value means that they are represented far apart.

Other distance choices answer slightly different questions:

- Euclidean distance, $d_{\text{Euc}}(r_i, r_j) = \|r_i - r_j\|_2$, keeps absolute response magnitude and is therefore sensitive to scaling.
- Mahalanobis distance, $d_{\text{Mah}}(r_i, r_j) = (r_i - r_j)^\top \Sigma^{-1} (r_i - r_j)$, downweights noisy or correlated response dimensions through the covariance matrix $\Sigma$.
- Rank-based distances ignore exact metric values and ask whether the ordering of near and far stimulus pairs is preserved.

So an RDM is not just a visualization. It is a precise mathematical summary of the geometry induced by a representation on a fixed stimulus set.

#### Comparing a brain RDM with a model RDM

The second step is to compare two RDMs. Let $D^{\text{brain}}$ be the RDM from a cortical area and $D^{\text{model}}$ be the RDM from a network layer. Because both matrices are symmetric and have zeros on the diagonal, one usually vectorizes only the upper triangle:

$$v(D) = (D_{12}, D_{13}, \dots, D_{1n}, D_{23}, \dots, D_{n-1,n}).$$

The RSA score can then be written as

$$\rho_{\text{RSA}} = \operatorname{corr}\big(v(D^{\text{brain}}), v(D^{\text{model}})\big).$$

In practice this is often a Spearman rank correlation, because researchers may care more about whether the model preserves the ordering of representational distances than about matching every numerical distance exactly.

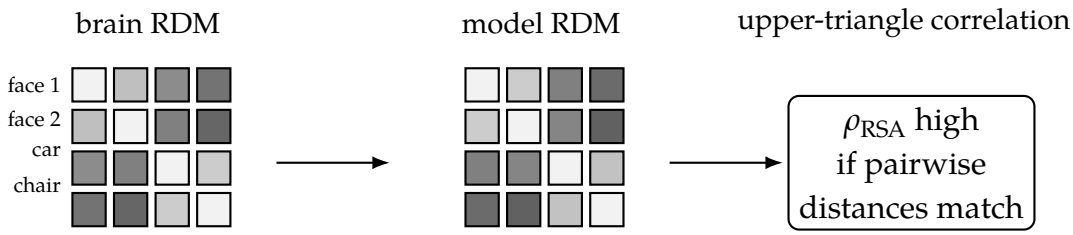


Figure 2.3: RSA is a second-order comparison. It does not ask whether one model unit equals one neuron. It asks whether pairs of stimuli that are close or far apart in cortex are also close or far apart in the model.

This is the main conceptual advantage. The number of units in the two systems can be different, and their axes need not correspond. A cortical area might be measured with $p = 500$ voxels and a network layer might have $q = 4096$ features. RSA can still compare them because both are reduced to the same $n \times n$ stimulus geometry.

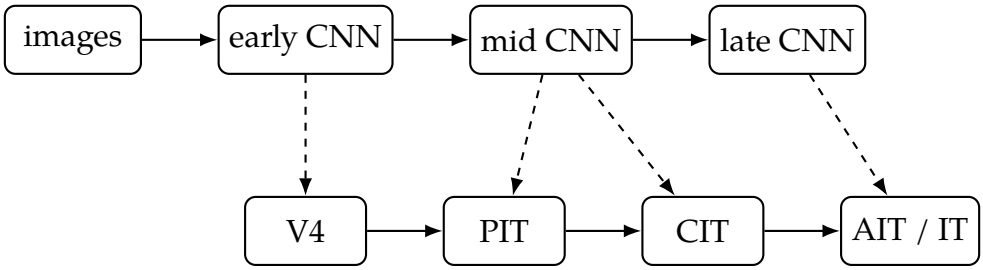


Figure 2.4: The comparison strategy highlighted in the lecture. Rather than matching units one by one, researchers compare which network layers best predict or resemble activity in successive ventral-stream areas.

#### How to interpret an RSA result

An RSA match should be read as evidence about representational geometry, not as a claim that the mechanisms are identical. A high RSA score says that the two systems place the tested stimuli in a similar pattern of relative distances. It does not by itself say that the same features were learned, that the same learning rule was used, or that the same computations occur internally.

There are two practical cautions:

- The stimulus set matters. An RDM built from faces, animals, tools, and places tests a different geometry than an RDM built only from small pose changes of one object.
- The noise ceiling matters. If repeated brain measurements are themselves only moderately correlated, then no model can be expected to reach a perfect RSA score.

This is why the lecture treats RSA as a powerful but limited bridge between deep learning and neuroscience. It gives a mathematically clean way to ask whether a model layer has the same *shape of representation* as a cortical area, while still leaving open the mechanistic question of how that representation is produced.

#### What the comparisons show

The lecture highlights a consistent empirical pattern: supervised deep models tend to explain the geometry of IT responses better than unsupervised alternatives, and models that are better optimized for object recognition often provide better predictions of neural data. This does not mean that cortex literally implements AlexNet or ResNet. It means that solving the right task appears to push artificial systems toward internal representations that partially resemble those found in the brain.

That is a strong scientific claim. It suggests that task demands and representational constraints are deeply connected: if a system must solve invariant object recognition well, some aspects of its internal geometry may be forced into a similar shape, whether the system is biological or artificial.

### 2.0.6 Deep networks as experimental tools

The lecture goes further than model comparison and treats deep networks as practical tools for neuroscience experiments. The Bashivan-style examples make this especially clear. The logic is:

1. train a convolutional network on a large visual task,
2. learn a mapping from model features to recorded neural responses,
3. optimize the input image to maximize the predicted response of a chosen neuron or population,
4. and then test the synthesized image experimentally.

This matters because a differentiable model can be used not only to *explain* old data but also to *design* new stimuli. If the model captures something important about a neuron's

preferred feature geometry, then the optimized image should strongly drive that neuron. The lecture discusses V4 experiments in which CNN-optimized images indeed strongly activated recorded neural sites.

So the deep model becomes more than a classifier. It becomes a hypothesis generator for what features a neural population is actually sensitive to.

### 2.0.7 What deep networks still miss

The lecture closes with several cautions. Modern deep networks are highly effective at image classification, but that does not make them complete theories of biological vision.

#### Adversarial fragility

Deep networks can be fooled by adversarial perturbations: small image changes that are nearly imperceptible to humans but cause a dramatic change in the predicted label. This shows that strong benchmark performance does not automatically imply human-like robustness or understanding.

#### Biological caveats

Several ingredients of standard deep learning remain biologically questionable:

- exact backpropagation,
- extensive supervised labels,
- strict convolutional weight sharing,
- and mostly feedforward inference in systems whose biological counterparts are strongly recurrent.

These caveats do not make deep networks unhelpful. They simply define the proper level of interpretation. Deep networks are useful computational models of visual recognition, and they may capture important aspects of cortical representation, but they are still abstractions.

### 2.0.8 Summary of concepts introduced in Lecture 1

The lecture introduced several ideas that should be kept distinct:

| Concept | Role in the lecture |
|---|---|
| Invariant object recognition | keep identity stable across natural image changes |
| Disentangling | transform tangled image manifolds into easier features |
| ImageNet | large-scale benchmark that made modern comparison possible |
| Convolution | local receptive fields plus weight sharing for images |
| Backpropagation | practical credit-assignment method for deep networks |
| RSA / RDMs | compare vectorized stimulus-distance matrices across brains and models |
| CNN-optimized stimuli | use models to generate experimental probes |
| Adversarial examples | reveal limits of current deep-network robustness |

### 2.0.9 Conceptual takeaways from Lecture 1

The opening vision lecture establishes the framework used later in the course.

- The main difficulty of vision is not detecting light, but constructing a representation that supports stable object recognition.
- Raw images are a bad format for recognition because nuisance variables strongly distort pixel values.
- Deep networks matter because they provide a concrete way to learn hierarchical feature spaces that partially solve this representation problem.
- The success of deep learning depends on a combination of architecture, data, optimization, and compute rather than on one isolated trick.
- Model evaluation can proceed on several levels at once: recognition performance, similarity to cortical representations, and usefulness for generating new experiments.
- High benchmark accuracy is not the same as biological plausibility or human-like robustness.

These points motivate the broader theme of the course: higher brain function can often be studied by asking what representation a system needs in order to solve a demanding cognitive task.

### Sources mentioned in Lecture 1


- J. J. DiCarlo and D. D. Cox (2007), "Untangling invariant object recognition," *Trends in Cognitive Sciences*.
- A. Krizhevsky, I. Sutskever, and G. E. Hinton (2012), "ImageNet classification with deep convolutional neural networks," *NeurIPS*.
- K. He, X. Zhang, S. Ren, and J. Sun (2016), "Deep residual learning for image recognition," *CVPR*.
- N. Kriegeskorte, M. Mur, and P. Bandettini (2008), "Representational similarity analysis," *Frontiers in Systems Neuroscience*.
- S.-M. Khaligh-Razavi and N. Kriegeskorte (2014), "Deep supervised, but not unsupervised, models may explain IT cortical representation," *PLoS Computational Biology*.
- D. L. K. Yamins and J. J. DiCarlo (2016), "Using goal-driven deep learning models to understand sensory cortex," *Nature Neuroscience*.
- P. Bashivan, K. Kar, and J. J. DiCarlo (2019), "Neural population control via deep image synthesis," *Science*.
- I. J. Goodfellow, J. Shlens, and C. Szegedy (2015), "Explaining and harnessing adversarial examples," *ICLR*.

# Mathematical Synthesis

Higher brain function is difficult because the variables of interest are latent. We observe images, actions, response times, choices, spike trains, and population activity; we infer beliefs, memories, task states, goals, and decisions. A good model makes that inference problem explicit.

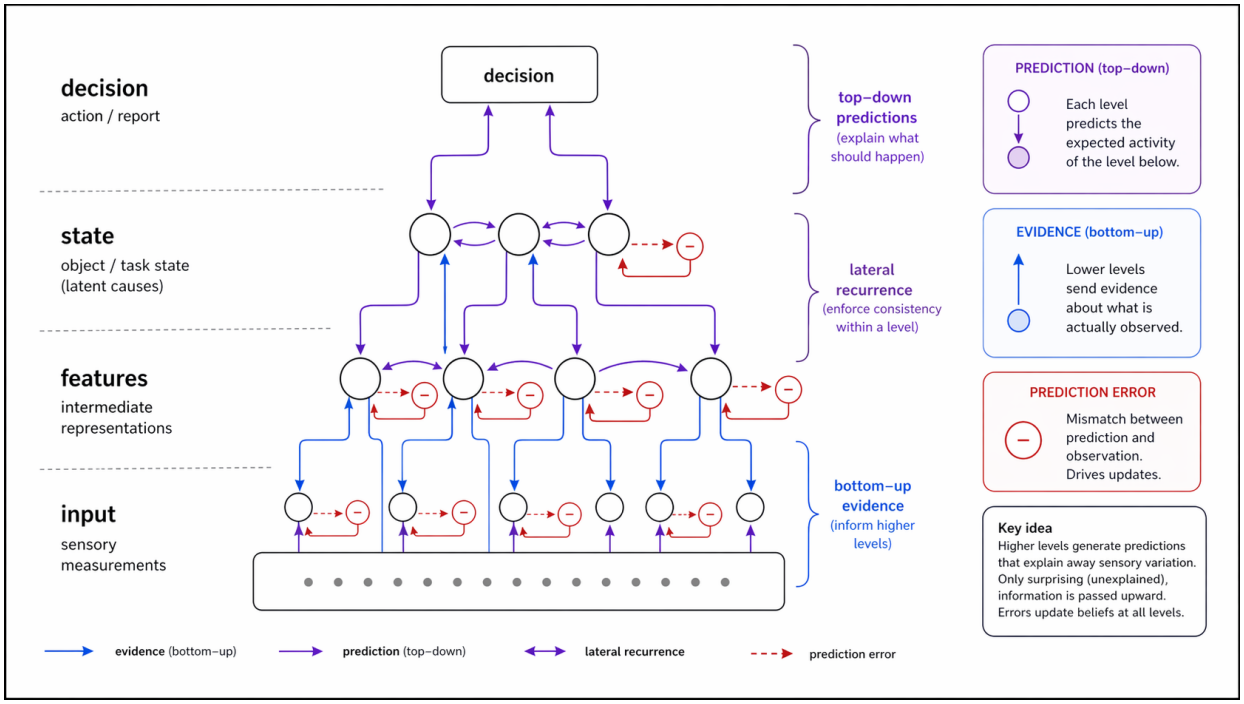


Figure 5: A compact view of hierarchical inference. Bottom-up evidence constrains higher latent states, top-down predictions explain expected sensory structure, lateral recurrence enforces consistency within a level, and prediction errors update the representation.

## Latent States

The basic abstraction is a state-space model:

$$z_{t+1} \sim p(z_{t+1} \mid z_t, a_t), \qquad x_t \sim p(x_t \mid z_t), \qquad a_t \sim \pi(a_t \mid z_t).$$

Here $x_t$ is observed sensory or behavioral data, $z_t$ is the latent cognitive state, and $a_t$ is an action or report. The model says that cognition is not a direct label attached to the stimulus; it is an inferred state that evolves over time and helps predict both future observations and behavior.

> Modeling intuition
>
> The latent state should be just rich enough to make the future conditionally simpler. If knowing $z_t$ makes past details irrelevant for predicting the next observation or action, then $z_t$ has captured the right memory for the task.

## Prediction And Error

In a hierarchical model, higher levels send predictions downward and lower levels send mismatches upward. A minimal predictive-coding objective can be written as

$$E(z) = \frac{1}{2}\|x - g(z)\|^2_{\Sigma_x^{-1}} + \frac{1}{2}\|z - \mu\|^2_{\Sigma_z^{-1}}.$$

The first term penalizes sensory prediction error. The second term penalizes deviation from prior expectations. Inference changes $z$ to reduce the total error:

$$\dot{z} \propto -\nabla_z E(z).$$

This equation is not just an optimization rule. It gives a mechanistic story for recurrent cortical dynamics: activity settles toward a state that jointly respects sensory evidence and top-down expectations.

## Representations As Coordinates

Perception becomes useful when the representation separates task-relevant variation from nuisance variation. If $x$ is an image and $z = f_\theta(x)$ is a representation, then a downstream decision should be simpler in $z$-space than in pixel space:

$$y \approx h(z), \qquad z = f_\theta(x).$$

Deep networks make this idea concrete. Early layers preserve local structure, intermediate layers combine features, and later layers tend to align more strongly with object or task variables. The scientific question is not only whether the model classifies correctly, but whether its internal geometry matches the invariances and error patterns of biological perception.

## Decisions Couple Belief To Action

Higher-brain models often end with a decision rule. A common normative form is

$$a^* \in \arg\max_a \mathbb{E}\left[U(a, Z) \mid x_{1:t}\right].$$

The action is chosen under a posterior belief about the latent state $Z$. This is the bridge from perception to behavior: the system does not act on the raw stimulus, but on an uncertainty-weighted internal estimate of what the stimulus means for the task.

# Bibliography


[DA01] Peter Dayan and L. F. Abbott. *Theoretical Neuroscience*. MIT Press, 2001.

[ORe+06] Randall C. O'Reilly, Yuko Munakata, Michael J. Frank, and Thomas E. Hazy. *Biologically Based Computational Models of High-Level Cognition*. Cambridge University Press, 2006.

[Wan08] Xiao-Jing Wang. "Decision making in recurrent neuronal circuits". In: *Neuron* 60.2 (2008), pp. 215–234.